

Análisis Numérico

Trabajo Práctico 4

Segundo cuatrimestre 2026

Instrucciones:

- Fecha de presentación: 10/09/26.
- Los grupos se conforman de 3 o 4 personas.
- Utilice Python como lenguaje de programación.
- Elabore un informe lo mas detallado posible, mencionando los problemas con los que se encontró intentando obtener las respuestas a las consignas.
- Enviar por correo un archivo comprimido único, **el informe en formato .pdf** y cualquier otro archivo que considere útil, como códigos u otros.
- Elaborar un video de no más de 3 minutos de duración sobre los aspectos más importantes del proceso y las conclusiones del trabajo y adjuntarlo al correo o incluir el link en el informe.

Archivos de Trabajo Proporcionados

Para el desarrollo del presente trabajo práctico, se proveen los siguientes archivos de datos:

- `vuelo_dron.mp4`: Video recortado grabado desde la cámara a bordo de la aeronave durante el vuelo original.
- `mapa_satelital_completo.jpg`: Imagen de alta resolución de la zona de vuelo.

Librerías Permitidas en Python

- CV2
- numpy
- matplotlib

Introducción Escenario de Misión

En la imagen satelital proporcionada (`mapa_satelital_completo.jpg`) se observa el campus de la Universidad Tecnológica Nacional – Facultad Regional Santa Fe y sus alrededores. Un piloto de dron intenta realizar un mapeo fotogramétrico automático de la zona siguiendo una ruta de inspección programada. La aeronave despegue y debe aterrizar exactamente en el mismo punto de control inicial.

Sin embargo, debido a interferencias de vibración en el gimbal de la cámara, ráfagas de viento e imperfecciones en el bucle de control, la aeronave presenta severas fluctuaciones en su velocidad e incertidumbre en la altitud de vuelo. El resultado directo de esta captura visual se encuentra grabado en el archivo `vuelo_dron.mp4`.

Su objetivo como equipo de ingeniería es procesar la información visual del video, diagnosticar las fallas cinemáticas, modelar la trayectoria real mediante métodos de interpolación numérica y re-sintetizar el vuelo optimizado.

Se incluye al final de este documento un Anexo A con la formalización teórica del algoritmo de Lucas-Kanade a modo orientativo para el desarrollo del trabajo.

1 Extracción por Flujo Óptico y Diagnóstico Cinemático

Extraiga la trayectoria del dron en todas las direcciones espaciales a partir del video ruidoso (`vuelo_dron.mp4`).

Grafique la trayectoria 2D en el plano (X vs Y) y la evolución de la escala de altitud/zoom relativo $Z(t)$ en función del tiempo t . Para la extracción espacial, explore el algoritmo de *Lucas-Kanade* determinando el flujo óptico de la cámara.

¿Con qué método de interpolación de los estudiados en la materia se relaciona internamente este algoritmo de visión? Explique y describa su funcionamiento teórico en relación a dicho método (ver Anexo A).

Comente como eligió los parámetros computacionales, qué función cumple cada uno, y su importancia relativa en el contexto de este problema particular. Realice un diagrama de bloques que describa las diferentes etapas del algoritmo.

Grafique la rapidez escalar instantánea obtenida mediante un método de diferenciación. Justifique por qué la derivada numérica discreta amplifica el ruido presente en los datos posicionales.

2 Modelado de Trayectoria

A partir de la información extraída en el punto anterior, seleccione un conjunto de nodos representativos distribuidos a lo largo del vuelo. Implemente un **método global** y un **método local por tramos** para recalcular las funciones de posición $X(t)$, $Y(t)$ y $Z(t)$. Grafique sobre la imagen satelital y en un plano 2D la comparación de ambas reconstrucciones junto con los datos ruidosos medidos. Realice una comparación crítica entre ambos métodos con el mayor detalle posible.

3 Síntesis y Renderizado de Vuelo Filtrado

Seleccione el método de interpolación que haya determinado como más conveniente en el inciso 2 y reprocese la trayectoria para renderizar un nuevo video suave de la aeronave utilizando la imagen satelital de alta resolución.

- a) **Caso A (Zoom Variable y Rapidez Constante):** Mantenga el perfil de altura/zoom $Z(t)$ del video original, pero reparametrice la trayectoria en el espacio para que la rapidez escalar del dron sea **estrictamente constante** ($v(t) = v_0$).
- b) **Caso B (Zoom Constante y Rapidez Constante):** Fije una altitud/zoom constante ($Z = 1.0$) y reparametrice la posición en el plano (X, Y) para lograr una rapidez constante v_0 .

Aplice nuevamente el algoritmo de flujo óptico (inciso 1) sobre sus propios videos renderizados. Grafique las trayectorias y los perfiles de velocidad obtenidos. Extraiga conclusiones teóricas sobre los inconvenientes físicos y cinemáticos que detecta al forzar a la aeronave a volar a rapidez constante en curvas cerradas.

4 Modelado de Condiciones Reales de Vuelo

Investigue cuáles son las restricciones y perfiles cinemáticos reales utilizados en los sistemas de control de autopilotos para drones comerciales.

Diseñe y renderice un último video que replique estas condiciones físicas reales. Determine experimentalmente las trayectorias y velocidades resultantes mediante su script de flujo óptico, justificando en el informe al detalle cada parámetro físico y numérico tenido en cuenta.

A The Tracking Problem: formalización del algoritmo de Lucas-Kanade

A.1 Planteo del problema (*The Tracking Problem*)

Sea $F(\mathbf{x})$ la intensidad de la imagen en el instante t (frame de referencia) y $G(\mathbf{x})$ la intensidad de la imagen en el instante $t + \Delta t$ (frame siguiente), con $\mathbf{x} = (x, y)$ las coordenadas de un píxel dentro de una ventana o *patch* W centrada en un punto de interés.

El problema de tracking consiste en hallar el vector de desplazamiento $\hat{\mathbf{u}} = (u, v)$ que mejor explica el movimiento aparente del patch entre ambos frames, es decir, tal que:

$$G(\mathbf{x}) \approx F(\mathbf{x} + \mathbf{u}) \quad \forall \mathbf{x} \in W \quad (1)$$

Una forma directa de resolver esto sería desplazar el patch de referencia sobre el frame en $t + \Delta t$ probando todos los valores posibles de $\hat{\mathbf{u}}$ y quedarse con aquel que produzca el menor error de coincidencia entre ambos patches. En la práctica esta igualdad nunca se cumple de forma exacta (ruido, cambios de iluminación, deformación local del patch), por lo que se busca $\hat{\mathbf{u}}$ como el minimizador de la **suma de diferencias al cuadrado** (*Sum of Squared Differences*, SSD):

$$E(\mathbf{u}) = \sum_{\mathbf{x} \in W} [G(\mathbf{x}) - F(\mathbf{x} + \mathbf{u})]^2, \quad \hat{\mathbf{u}} = \arg \min_{\mathbf{u}} E(\mathbf{u}) \quad (2)$$

Este enfoque de fuerza bruta (búsqueda exhaustiva de $\mathbf{u}$) tiene un costo computacional alto: para cada punto a rastrear, en cada frame, habría que evaluar $E(\mathbf{u})$ para todo un rango de desplazamientos candidatos. Esto vuelve al método impracticable para las demandas de un sistema de video en tiempo real.

A.2 La mejora: algoritmo de Lucas-Kanade

Lucas y Kanade (1981) plantean una forma mucho más eficiente de resolver este mismo problema, evitando la búsqueda exhaustiva. La idea central parte de asumir que el desplazamiento del patch entre dos frames consecutivos es suficientemente pequeño, de modo que $F(\mathbf{x} + \mathbf{u})$ pueda aproximarse mediante su desarrollo de Taylor de primer orden alrededor de $\mathbf{u} = 0$:

$$F(\mathbf{x} + \mathbf{u}) \cong F(\mathbf{x}) + \frac{\partial F^T}{\partial \mathbf{x}} \mathbf{u} \quad (3)$$

Sustituyendo esta aproximación en $E(\mathbf{u})$, el error deja de depender de $\mathbf{u}$ de forma arbitraria y pasa a ser una función cuadrática de $\mathbf{u}$:

$$E(\mathbf{u}) \approx \sum_{\mathbf{x} \in W} \left[G(\mathbf{x}) - F(\mathbf{x}) - \frac{\partial F^T}{\partial \mathbf{x}} \mathbf{u} \right]^2 \quad (4)$$

Al ser cuadrática, este error tiene un único mínimo, que puede obtenerse de forma cerrada y explícita (sin necesidad de probar valores candidato uno por uno):

$$\mathbf{u} = \sum_{\mathbf{x} \in W} (G(\mathbf{x}) - F(\mathbf{x})) \frac{\partial F}{\partial \mathbf{x}} \left[\sum_{\mathbf{x} \in W} \frac{\partial F}{\partial \mathbf{x}} \frac{\partial F^T}{\partial \mathbf{x}} \right]^{-1} \quad (5)$$

Esta es la clave de la eficiencia del método: transforma un problema de búsqueda en un problema de álgebra lineal, resoluble con un puñado de sumas sobre el patch y la inversión de una matriz de 2×2 .

A.3 Formulación matricial — sistema sobredeterminado y solución cerrada

La expresión anterior es, en esencia, la solución de un sistema de ecuaciones lineales sobredeterminado resuelto por **mínimos cuadrados**. Si se llama A a la matriz cuyas filas son los gradientes espaciales $\partial F / \partial \mathbf{x}$ evaluados en cada píxel del patch, y $\mathbf{b}$ al vector de diferencias de intensidad $G(\mathbf{x}) - F(\mathbf{x})$, el planteo se resume como:

$$A \mathbf{u} = \mathbf{b} \quad \longrightarrow \quad \mathbf{u} = (A^T A)^{-1} A^T \mathbf{b} \quad (6)$$

Con $A \in \mathbb{R}^{n \times 2}$ y $n \gg 2$ (hay muchos más píxeles en el patch que incógnitas), el sistema es incompatible en general, y $(A^T A)^{-1} A^T \mathbf{b}$ es exactamente la solución de mínimos cuadrados dada por las **ecuaciones normales** — la misma técnica utilizada en la Sección 2 de este TP para el ajuste polinomial global. La matriz $\mathcal{M} = A^T A$ se conoce como **tensor de estructura** del patch, y es la que determina si el sistema tiene o no una solución numéricamente confiable.

A.4 Consideraciones en la elección de los puntos a rastrear y pre-tratamiento

Que el sistema anterior tenga solución cerrada no garantiza que esa solución sea confiable: todo depende de si la matriz $\mathcal{M} = A^T A$ es invertible en un sentido numéricamente sano, y eso depende directamente de qué parte de la imagen se eligió como patch.

- Si el patch cae sobre una **zona plana** (sin textura, gradientes casi nulos), $\mathcal{M}$ resulta prácticamente singular: el sistema queda mal condicionado y cualquier ruido de intensidad produce estimaciones de $\mathbf{u}$ erráticas.
- Si el patch cae sobre un **borde o línea recta**, hay gradiente fuerte solo en una dirección: el sistema queda indeterminado en la dirección tangente al borde (*problema de apertura*), y solo puede estimarse con confianza la componente del desplazamiento perpendicular al borde.
- Si el patch contiene una **esquina** (gradientes fuertes en dos direcciones independientes), $\mathcal{M}$ queda bien condicionada y el desplazamiento puede estimarse de forma estable en ambas direcciones.

Esta es la razón por la cual, antes de correr Lucas-Kanade, se aplica un criterio de selección de puntos (Shi-Tomasi, `cv2.goodFeaturesToTrack`) que descarta zonas planas y bordes, y prioriza esquinas: no es un paso independiente del algoritmo, sino un pre-tratamiento que garantiza que la matriz que Lucas-Kanade va a invertir esté en condiciones de dar una solución confiable.

A esto se suma la hipótesis de partida de la Sección A.2 — que $\mathbf{u}$ sea “suficientemente pequeño” para que la aproximación de Taylor sea válida —, que en la práctica no siempre se cumple si el movimiento entre frames es grande. Por eso se recurre a una **pirámide de resolución** (`maxLevel`): se estima primero el desplazamiento en una versión reducida de la imagen y se refina progresivamente en las escalas más finas. El tamaño de ventana (`winSize`) es, a su vez, un compromiso: una ventana más grande da más ecuaciones al sistema (mayor robustez frente al ruido), pero también aumenta el riesgo de violar la hipótesis de desplazamiento uniforme dentro del patch.